

A Mistake-Based Personalised Question recommender System

Anonymous

Abstract

The proliferation of online education platforms has generated massive amounts of student behavioral data. However, a core industry challenge remains: how to leverage this data to solve the problems of "ineffective practice" and low learning efficiency. Traditional recommend systems often aim to adapt difficulty or reinforce memory, overlooking the critical role of accurate self-diagnosis in effective learning. To address this challenge, this study proposes and validates a novel personalized question recommend system oriented around "error prediction." The system aims to guide students to confront their knowledge weaknesses and achieve efficient, targeted learning by proactively identifying and recommending questions they are most likely to answer incorrectly.

This research is based on the large, public educational dataset XES3G5M. A rigorous data preprocessing pipeline was designed, which included: focusing on the more cognitively demanding "思维拓展" (Mind Expansion) module, filtering for active users with over 120 interactions within a 90-day span, and aggregating fine-grained knowledge concepts into 7 high-level modules to construct a high-quality sequential dataset. In terms of methodology, we used the classic Deep Knowledge Tracing (DKT) model as a baseline and developed an advanced model, ADGKT, which integrates a multi-head self-attention mechanism to more accurately capture students' dynamic knowledge states.

To scientifically evaluate the recommend system's true efficacy, this study innovatively employed a "Fixed Threshold Hit Strategy" for sequential validation. By setting a strict predicted probability threshold ($\theta=0.1$), this strategy assesses the precision, coverage, and frequency of the system's high-confidence "weakness diagnoses." On an independent cold-start test set, the experimental results were outstanding: the system achieved a Hit Success Rate around 75%, meaning nearly three-quarters of the recommended weak-point questions were indeed the students' knowledge gaps. The User Coverage Rate was nearly 90%, indicating the system can provide effective diagnoses for the vast majority of users. Furthermore, an average of 8.7 hits per user ensures the service's continuity and utility.

In conclusion, this research not only confirms the immense potential and feasibility of the "learning from mistakes" recommendation paradigm through offline experiments but also provides a clear, viable technical path and theoretical basis for how online education products can achieve a deeper, more diagnostically valuable level of personalization.

Keywords: *Knowledge Tracing; Personalized Recommendation; Error Prediction; Deep Learning; Online Education*

Chapter 1: Introduction and Problem Definition

1.1 Research Background: The "Ineffective Learning" Dilemma in Online Education

With the global wave of digital education, online learning platforms have become an indispensable infrastructure of modern education. The massive, high-dimensional student behavior data they accumulate offers unprecedented opportunities for personalized education. However, most platforms still face a core challenge in practice: how to translate data potential into real learning efficiency. Mainstream content recommendation strategies, such as global recommendations based on course popularity or linear path planning based on fixed knowledge graphs, are essentially "broadcast" or "homogenized" teaching models. This approach fails to fully mine and respond to the significant individual differences among learners, often leading students into a predicament of "ineffective learning."

A typical manifestation of "ineffective learning" is "ineffective practice." During exam preparation or review, students tend to repeatedly practice question types they have already mastered. While this behavior can bring a fleeting sense of accomplishment, it does not effectively expand their skill boundaries. The root cause lies in students' biased perception of their own knowledge systems, a phenomenon known as the "Illusion of Competence." [1]. They find it difficult to accurately and objectively identify their true knowledge gaps and weaknesses. Consequently, a significant disconnect emerges between the time invested in learning and the growth of learning outcomes, with a great deal of valuable time being consumed by low-value, repetitive tasks.

1.2 Research Motivation:

A Paradigm Shift from "Difficulty Adaptation" to "Error Prediction" [6][5]. To break this impasse, this study posits that the core value of a truly efficient intelligent education system lies not only in "giving a person a fish" (providing content) but also in "teaching them how to fish" (cultivating learning strategies). Accurate self-diagnosis is the logical starting point for any effective learning strategy. Based on this, we propose

a novel recommender system paradigm oriented around "Error Prediction."

The core philosophy of this paradigm is: **for a learner, the most diagnostically valuable exercises are those most likely to expose their knowledge gaps.** Unlike traditional systems that strive to recommend moderately difficult questions, this system aims to proactively and precisely identify and recommend the questions that students are most likely to get wrong. By guiding students out of their learning "comfort zones" to confront the potential flaws in their knowledge systems, the system helps them achieve the most efficient targeted learning, directing resources precisely to the key points of skill improvement. This "learning from mistakes" philosophy aims to transform every error into a valuable learning opportunity, thereby maximizing the return on learning investment.

1.3 Competitor Analysis and System Positioning

To clarify the innovation and unique positioning of this research, we selected the global language learning giant **Duolingo** and the representative of China's K-12 online education, **Yuanfudao**, as reference points for in-depth analysis.

1.3.1 Case Study: Duolingo

- **Core Technology & Pedagogy:** Duolingo's core technology revolves around its **"Birdbrain" AI system**. Based on the **Spaced Repetition System (SRS)** theory, this system analyzes massive amounts of answer data from hundreds of millions of users worldwide to dynamically calculate the "half-life" (i.e., the forgetting curve) for each knowledge point (like words or grammar) for every user. Its recommendation logic is to precisely push relevant exercises to reinforce memory just as the user is about to forget a knowledge point[2]. This method is essentially **"memory-consolidation-oriented,"** aiming to optimize review efficiency and combat forgetting.

- **Business Model & Revenue:** Duolingo employs a typical **Freemium model**. Free users contribute data and watch ads, while the paid subscription service, "Super Duolingo," offers ad-free experiences, unlimited "hearts" (lives), and other premium features. According to its financial reports, the company's revenue exceeded \$500 million in 2023, with subscription fees being its primary source of income.

- **Positioning:** Duolingo's success lies in its highly gamified experience and personalized review based on memory theory. Its recommendation goal is to help users **"memorize."**

1.3.2 Case Study: Yuanfudao

- **Core Technology & Pedagogy:** Yuanfudao's technical barrier is its "**AI-driven large-scale personalized tutoring.**" Its system includes not only live interactive technology supporting massive concurrency but, more importantly, a series of AI applications behind it, such as using computer vision to automatically grade and explain problems and an intelligent Q&A system based on natural language processing. In terms of exercise recommendations, Yuanfudao focuses more on assessing mastery of knowledge points. By analyzing students' historical answer records, it matches them with questions of appropriate difficulty, drawing upon the pedagogical concept of the 'Zone of Proximal Development' (ZPD) where learners are supported to tackle challenges slightly beyond their current independent ability [6]. Its logic is 'difficulty-adaptive.' ... Positioning: Yuanfudao's system aims to simulate the tutoring process of an excellent teacher, embodying the principle of 'scaffolding' support described by Wood, Bruner, and Ross [6], achieving large-scale individualized instruction through technology. Its recommendation goal is to help users 'learn.'"

- **Business Model & Revenue:** Yuanfudao's main income comes from **tuition fees for online courses**. As an unlisted unicorn company, its valuation once exceeded \$15 billion at its peak, with annual revenues in the billions of dollars, reflecting the market's high recognition of its technology-driven teaching services.

- **Positioning:** Yuanfudao's system aims to simulate the tutoring process of an excellent teacher, achieving large-scale individualized instruction through technology. Its recommendation goal is to help users "**learn.**"

1.3.3 Differentiated Positioning of Our System

Compared to the competitors mentioned above, the "error prediction" system proposed in this study has a distinctly different positioning, as shown in the table below:

Feature	Duolingo	Yuanfudao	This Project's System (Error-Centric)
Core Logic	Spaced Repetition, Anti-Forgetting	Difficulty Adaptation, Steady Progress	Error Prediction, Exposing Gaps
Recommendation Goal	Recommend what the user is about to forget	Recommend challenging but solvable questions	Recommend questions the user is most likely to get wrong

Pedagogical Philosophy	Gamification-driven, Memory Consolidation	Simulating excellent teachers, Individualized Instruction	Using mistakes as a mirror, Precise Diagnosis
Applicable Scenarios	Fragmented-time language learning	K-12 synchronous tutoring, Systematic learning	Pre-exam sprints, Efficient review, Self-diagnosis

In summary, this system is not intended to replace the former two but to provide a more targeted solution in specific scenarios. It is particularly suitable for students who have a certain foundation but wish to quickly identify and solve their own problems.

1.4 System Interaction and Problem Definition

1.4.1 System Deployment and Interaction Flow

This recommender system is designed as an independent functional module that can be deployed on any online learning platform (Web or mobile), for instance, as an "AI Weakness Diagnostic Center." Its core interaction flow is as follows:

- **User profiling:** The number of times users answer various questions in different modules and their accuracy rates.
- **Cold start:** Recommend questions that users are most likely to answer correctly in order to obtain their basic interaction data.
- **User Inputs:** The system's primary data source is the user's **implicit behavioral sequence** i.e., all their historical question-answering records on the platform, including question IDs, knowledge points, correctness of answers (correct/incorrect), timestamps, etc.
- **Recommendations Provided:** The system generates a personalized "Weakness Breakthrough" question list for the user, consisting of the Top-K questions that the system predicts the user is **most likely to answer incorrectly**.
- **When to Recommend:**
 - **Active Trigger:** When the user completes a set of exercises or clicks a button like "Generate My Diagnostic Report" in their personal center.
 - **Passive Push:** When the user logs into the platform, a "Today's Challenge" is displayed on the personalized dashboard on the homepage, or it is pushed through a weekly learning report.
- **User Feedback & Iteration:**
 - The user's actions on the recommended questions (whether correct or incorrect) will serve

as new data points, appended to their personal historical sequence in real-time or near-real-time.

- This dynamically updated sequence will be used for the next recommendation calculation, forming an adaptive learning loop where the model's effectiveness continuously evolves with user engagement.

1.4.2 Formal Problem Definition

The core recommendation task in this study is formally defined as a **Sequence-based Binary Classification/Prediction Problem**[3].

Given a student's interaction history sequence up to time $t-1$, $S_u = \{(q_1, c_1, r_1), (q_2, c_2, r_2), \dots, (q_{t-1}, c_{t-1}, r_{t-1})\}$, where q_i is the question ID, c_i is the corresponding knowledge concept (or module) ID, and $r_i \in \{0, 1\}$ is the student's response (0 for incorrect, 1 for correct).

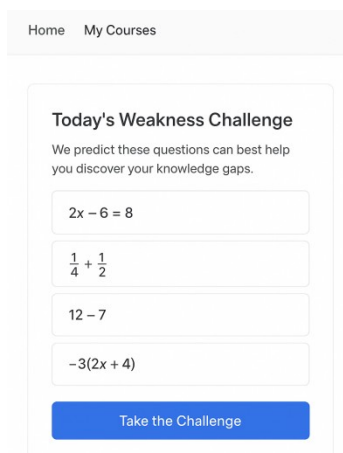
Our goal is to train a deep learning model M that can predict the probability of the student answering a subsequent (unseen) question q_t correctly, based on their history S_u and the question q_t (with its concept c_t). The model's output is a probability value:

$$P(r_t=1 | S_u, q_t, c_t)$$

where $P(r_t=1 | S_u, q_t, c_t)$ represents the probability that the model predicts the student will answer question q_t correctly. The recommender system's decision logic is based on the **inverse sorting** of this probability. For all candidate questions, the system calculates the above probability value and recommends the K questions with the lowest probabilities.

1.4.3 Conceptual Application and User Interface

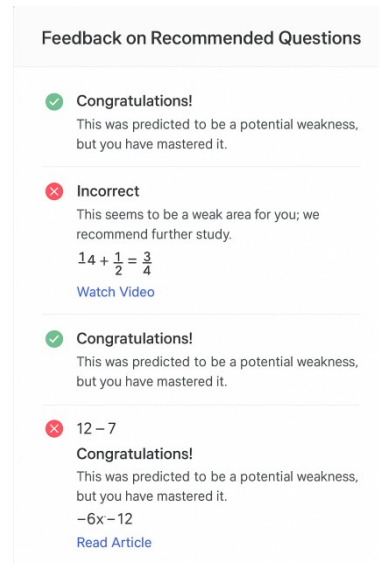
To intuitively demonstrate the application scenarios of the system, we have designed the following user interface (UI) mockups. This interface aims to seamlessly integrate the core functionality of "error prediction" into the user's learning flow.



• [Figure 1.1: Personalized Recommendation Module on the Learning Platform's

Homepage]

○ On the main interface after a user logs in, a module named "Today's Weakness Challenge" will be presented. This module directly displays 3-5 questions with high predicted error rates, selected by the model, accompanied by a brief explanation: "We predict these questions can best help you discover your knowledge gaps," to motivate the user to take on the challenge.



• [Figure 1.2: Instant Feedback and Guidance After Completing Recommended Questions]

○ When the user completes the challenge, the system provides immediate, detailed feedback. For incorrect answers, it not only provides the standard answer and explanation but also links to relevant knowledge point explanation videos or articles, with a prompt like "This seems to be a weak area for you; we recommend further study." For correct answers, it offers affirmation, stating, "Congratulations! This was predicted to be a potential weakness, but you have mastered it." This instant, targeted feedback reinforces the "learning from mistakes" experience, completing the learning loop.

Chapter 2: Dataset Analysis and Preprocessing

2.1 Data Source and Selection Rationale

This study uses the large-scale, public educational dataset **XES3G5M** as its experimental basis. This dataset originates from a real online math learning platform and completely records the learning trajectories of over 18,000 third-grade students over a period of time. The selection of this dataset is primarily based on the following considerations:

1. **Ecological Validity:** The data is generated in a real, uncontrolled learning environment, which can effectively reflect students' natural learning behavior patterns, ensuring the applicability of the research conclusions to real-world scenarios.

2. **Scale and Temporality:** The dataset is large in scale and strictly records students' interaction sequences with timestamps, providing a solid data foundation for training deep sequential models (such as knowledge tracing models).

3. **Structured Knowledge System:** The dataset provides a clear hierarchical structure of knowledge concepts (Knowledge Concept Hierarchy), which is crucial for content-aware modeling and feature engineering.

4. **Independent Cold-Start Evaluation Set:** The dataset specifically provides a test set (test.csv) with users who have no overlap with the training set, offering a standardized benchmark for evaluating the model's generalization ability in new user (i.e., cold-start) scenarios.

2.2 Data Preprocessing and Feature Engineering

Although the raw dataset is rich, it also contains a significant amount of noise and information not directly relevant to the research objectives. To construct a high-quality, high-signal-to-noise training corpus, we designed and implemented a rigorous multi-stage data preprocessing and feature engineering pipeline.

Stage 1: Domain Focusing and User Filtering

1. Defining the Research Scope - "Mind Expansion" Module:

- **Analysis and Decision:** By parsing the question metadata (questions.json), we constructed the dataset's knowledge concept hierarchy (as shown in Figure 2.1). The analysis revealed that "Mind Expansion" (思维拓展) is the top-level module with the broadest question coverage and the most comprehensive content, containing **6,940** unique questions. To ensure the depth of the research and the consistency of the data, we limited our research scope to this module and filtered out all question-answering records that did not belong to it.

- **Pros and Cons:**

- **Pros:** This allows the model to focus on a domain with higher cognitive demands and more complex knowledge relationships, thereby more effectively capturing deep learning patterns and avoiding interference from a large number of simple, memory-based questions. This enhances the research's depth and the model's diagnostic value.

- **Cons:** The model's direct applicability is limited to this specific module; its predictive ability for other modules is unknown. This is a typical trade-off between **research depth** and

model breadth.

2. Filtering for High-Activity Users:

- **Analysis and Decision:** The effectiveness of a knowledge tracing model is highly dependent on the length of the user interaction sequence; a sufficiently long sequence is a prerequisite for the model to learn effective patterns. After focusing on the domain, we conducted a detailed statistical analysis of user sequence lengths (see Figure 2.2). The distribution plot clearly shows that the vast majority of continuously active users have interaction counts concentrated above 100, forming a distinct main user group. Based on this data-driven observation, we set the **minimum interaction length threshold to 120**. This decision aims to focus the research on this main user group with common characteristics, filtering out noise that might be introduced by incidental behavior or exploratory clicks from users with very few interactions (e.g., users who churn after only a few attempts). This step reduced the number of effective users from 33,392 to **22,449**.

- **Pros and Cons:**

- **Pros:** This ensures that each sequence used for training contains enough contextual information for the deep learning model to learn meaningful temporal dependencies and the evolution of knowledge states, avoiding underfitting due to insufficient data. Focusing on the behavioral patterns of the core user group also helps the model learn more stable and representative patterns of knowledge transition.

- **Cons:** This approach discards data from new or low-activity users, making the cold-start problem for these user groups more prominent. The final trained model may not generalize well to users with fewer interaction behaviors.

Stage 2: Sequence Normalization and Time Window Constraints

1. Sequence Length Normalization:

- **Analysis and Decision:** To standardize the model input dimension and focus on the user's most recent learning state, we **truncated all user sequences to the latest 120 interactions**. This length is consistent with the filtering threshold from the previous step.

- **Pros and Cons:**

- **Pros:** This allows the model to handle fixed-length inputs, greatly simplifying model design and the complexity of batching, which improves training efficiency. Focusing on recent interactions also better reflects the user's current knowledge level, consistent with the cognitive principle that knowledge can be forgotten over time.

- **Cons:** For highly active users with interaction histories far exceeding 120, this truncation

might discard earlier but potentially still relevant learning history, such as the first encounter with certain fundamental concepts.

2. Applying a Time Window:

- **Analysis and Decision:** A student's knowledge state is dynamic and can decay due to forgetting. An interaction sequence spanning several months or even a year may have early records with limited reference value for predicting current performance. Therefore, we imposed a strict time window constraint, retaining only those users whose latest 120 interactions occurred within a **90-day period**. This crucial step ultimately filtered the dataset down to **10,382 high-quality user sequences**.

- **Pros and Cons:**

- **Pros:** This greatly purifies the data, ensuring that each sequence represents a coherent and effective learning period. It significantly improves the signal-to-noise ratio, allowing the model to learn a knowledge state that is closer to the user's "current" state and avoiding interference from "old knowledge."

- **Cons:** This might exclude users with more sporadic learning habits or longer learning cycles (e.g., users who only study once a weekend), thereby narrowing the applicable user population of the model.

Stage 3: Knowledge Concept Feature Engineering

- **Analysis and Decision:** The granularity of the original knowledge concepts is too fine (thousands of concepts), and using them directly would lead to a high-dimensional feature space and severe data sparsity. To address this, we used the knowledge concept path metadata (kc_routes) to **aggregate the fine-grained concept IDs upwards** into their parent sub-modules within the "Mind Expansion" domain, resulting in **7** higher-level categories.

- **Pros and Cons:**

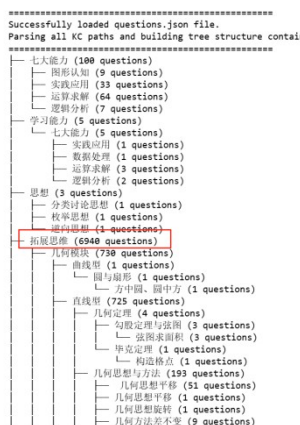
- **Pros:** This is an effective **dimensionality reduction** and **smoothing** technique. It significantly alleviates data sparsity and encourages the model to learn higher-order relationships between concepts (e.g., the relationship between "Geometry" and "Combinatorics") rather than memorizing thousands of isolated, fine-grained knowledge points. This greatly enhances the model's generalization ability and interpretability.

- **Cons:** It results in the loss of some detailed knowledge information. The model may not be able to make fine distinctions between questions that fall under the same sub-module but have significant differences in difficulty or focus. For example, the model might know a student is weak in "Geometry" but cannot distinguish whether the weakness is in "area calculation" or "solid figure

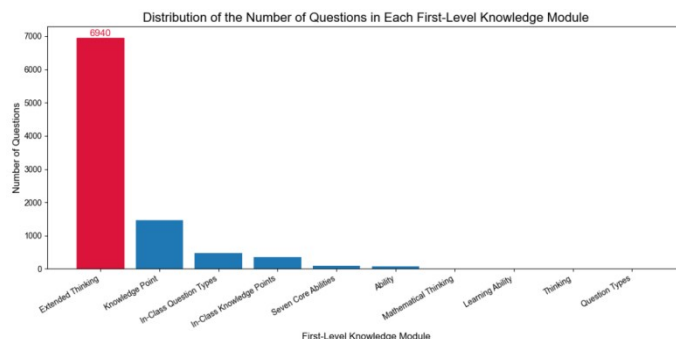
recognition."

2.3 Exploratory Data Analysis (EDA) Core Findings

During the data preprocessing, we conducted exploratory data analysis, and one key finding validated the feasibility of our research's core hypothesis. In the final constructed validation set, the overall average correctness rate for all students was approximately **80.4%**. However, for the questions that our initial model predicted would be answered incorrectly, the students' **actual error rate was as high as 75.3%**. This means that if a question is chosen at random, a student has about an 80% chance of getting it right; but after being filtered by our model, the recommended questions have nearly a 75% probability of being a student's true weak point. This significant difference (a baseline error rate of about 20% vs. a model-located error rate of 74.4%) intuitively demonstrates the immense potential and effectiveness of the "error prediction" strategy.



- [Figure 2.1: Visualization of the Dataset's Knowledge Concept Hierarchy]



- [Figure 2.2: Distribution of User Sequence Lengths in the "Extend Thinking" Module Before Filtering]

2.4 Dataset Suitability Assessment

Based on the above analysis, we assess the suitability of the XES3G5M dataset for this study:

Strengths:

- **High Ecological Validity:** The data comes from a real learning scenario, ensuring the real-world significance of the research conclusions.
- **Temporal Integrity:** The interaction data, strictly ordered by time, perfectly fits the requirements of sequential modeling tasks.
- **Structured Knowledge System:** A clear knowledge concept hierarchy provides a solid foundation for content-aware feature engineering.

Limitations:

- **Data Sparsity:** Even after filtering, the "student-knowledge point" interaction matrix remains sparse, and the cold-start problem is a challenge the model must address.
- **Missing Metadata:** The dataset lacks intrinsic properties of the questions (such as difficulty, discrimination) and detailed user profiles (such as prior academic level, learning style), which limits the feature dimensions the model can utilize.
- **Behavioral Data Noise:** The data may contain atypical learning behaviors (such as random guessing, someone else doing the work), and this noise could interfere with model training.

Chapter 3: Methodology

3.1 Overall Methodology and Model Selection

To address the sequential prediction problem defined in this study, we adopted **Knowledge Tracing (KT)** as our core methodology. Knowledge tracing aims to simulate the dynamic evolution of a student's knowledge mastery by analyzing their historical interaction sequences, thereby predicting their performance on future tasks. This aligns perfectly with our goal of predicting question correctness.

In this project, we implemented and compared two representative knowledge tracing models: the classic **Deep Knowledge Tracing (DKT)** as a baseline, and a more advanced model we selected as our core model[4]. **Attention-augmented Dynamic Graph Knowledge Tracing (ADGKT)**.

3.1.1 Baseline Model: DKT

DKT was one of the first models to apply deep learning, particularly Recurrent Neural Networks (RNNs), to the field of knowledge tracing. We chose DKT as a performance benchmark for the following reasons:

- **Representativeness and Simplicity:** As a pioneering work in the field, DKT has a clear structure (Embedding + LSTM), is easy to implement, and is a recognized standard for evaluating the performance of more complex models.

- **Efficiency:** Its architecture is relatively simple, leading to fast training speeds and low resource consumption, making it suitable as a baseline for quick iteration and comparison.

However, DKT also has inherent limitations, such as its inability to explicitly distinguish between the knowledge concepts of questions, its limited memory capacity for long sequences, and its lack of an attention mechanism to focus on key historical information. These shortcomings are what motivated us to introduce the ADGKT model.

3.1.2 Core Model: ADGKT

ADGKT makes several key improvements over DKT, making it more suitable for the task of this study. The reasons for choosing it lie in its three core advantages:

1. **Powerful Sequential Modeling Capability:** It retains the Gated Recurrent Unit (GRU) as a core component to effectively capture the temporal dependencies in the student's learning process, simulating patterns of mastery and forgetting.

2. **Dynamic Focusing on Key History:** It introduces a **Multi-Head Self-Attention** mechanism, which allows the model to dynamically assign weights to each interaction in the historical sequence when making a prediction. This means the model can automatically identify and focus on the historical records most relevant to the current prediction, greatly enhancing its ability to model long sequences.

3. **Rich Content Information Fusion:** ADGKT can simultaneously process three input streams: question IDs, module IDs, and response correctness. By explicitly fusing "what question the student did" with "what knowledge concept was learned," the model can achieve a deeper understanding of the content structure, thereby improving prediction accuracy and the interpretability of recommendations.

3.2 Model Architecture Details

3.2.1 DKT Architecture

Our DKT baseline model was implemented in the PyTorch framework, with the following core structure:

1. **Input Encoding:** A student's interaction is encoded as a single integer: $\text{interaction} = \text{question_id} * 2 + \text{is_correct}$. This integer is then mapped to a 128-dimensional vector via an Embedding Layer.

2. Sequential Modeling Layer (LSTM Layer): The sequence of encoded vectors is fed into a single-layer LSTM network (with a hidden size of 128) to capture temporal dependencies and model the evolution of the student's knowledge state.

3. Output Layer: The output from the LSTM at each timestep is passed through a Fully Connected Layer, which maps it to a vector with a length equal to the total number of questions.

4. Loss Function: We use the **Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss)**, which combines a Sigmoid activation and BCE loss calculation for better numerical stability. The loss is calculated only for the target questions that actually appeared in the sequence.

3.2.2 ADGKT Architecture

Our ADGKT model was also implemented using PyTorch. Its data flow and core components are illustrated below:

The hierarchical structure is detailed as follows:

1. Input Layer: It receives three parallel input sequences: question IDs (questions), module IDs (concepts), and response correctness (responses).

2. Embedding Layer:

- **Question Embedding (q_embed):** Maps discrete question IDs into a 128-dimensional vector space, capturing the unique features of each question.

- **Module Embedding (c_embed):** Maps the 7 high-level module IDs into a 128-dimensional vector space, representing the macroscopic features of different knowledge domains.

- **Response Embedding (r_embed):** Maps the binary responses (0 or 1) into a 128-dimensional space, allowing the model to learn the different impacts of "correct" and "incorrect" answers on the student's knowledge state.

3. Interaction Fusion Layer: This layer is key to information fusion. We concatenate the question and module embeddings, then fuse them into a unified "content embedding" via a fully connected layer. This embedding is then added to the response embedding of the corresponding step, forming a vector that comprehensively represents the complete event of "answering a specific question correctly/incorrectly within a certain knowledge module."

4. Attention Layer: The fused interaction sequence is passed through a LayerNorm and then a **Multi-Head Self-Attention** mechanism (with 4 attention heads). The attention mechanism calculates the importance of each interaction relative to all others in the sequence and performs a weighted sum accordingly. This allows the model to dynamically focus on the most important historical steps for the current prediction when dealing with long sequences.

5. Sequential Modeling Layer: The attention-weighted sequence is passed through another LayerNorm and then into a **GRU** network (with a hidden size of 128). The GRU is responsible for capturing long-term temporal dependencies in the sequence and updating the hidden state, which represents the student's current knowledge level.

6. Prediction Layer: The final hidden state from the GRU is fused with the embedding vector of the **target question**. This is then passed through a final fully connected layer and a Sigmoid activation function to output a probability value between 0 and 1, representing the model's prediction of the student's probability of answering the target question correctly.

3.3 Model Hyperparameter Selection and Discussion

To ensure the reproducibility of the experiments, we carefully selected the model hyperparameters. The below is detailed explanation of the key parameters and the rationale for their selection. Here's the table converted into a coherent paragraph format:

The model hyperparameters were carefully selected to optimize performance while maintaining computational efficiency. For general structural parameters, we set the embedding size to 128 dimensions, balancing expressive power and overfitting risk. This standard industry value provides sufficient capacity to capture subtle differences between questions and concepts without creating an excessive number of parameters that could cause overfitting on our limited dataset, unlike higher dimensions such as 256 or 512. The hidden size was also set to 128, consistent with the embedding layer, to balance memory capacity and computational cost. For RNN architectures like LSTM and GRU, the hidden size determines the capacity of the memory cell and how much information about past sequences can be stored, allowing for effective information propagation while maintaining reasonable computational overhead.

For ADGKT-specific parameters, we implemented 4 attention heads to achieve multi-perspective information aggregation. Multi-head attention enables the model to attend to information from different representation subspaces in parallel, meaning the model can examine historical data from four different angles simultaneously - for example, one head might focus on recent errors while another examines conceptual relationships. This configuration is key to enhancing the model's ability to capture complex dependencies while remaining computationally manageable. We applied a dropout rate of 0.2 as a core regularization technique to prevent overfitting. This randomly drops neuron outputs with 20% probability during training, forcing the network to learn more robust and redundant features rather than over-relying on specific neurons, significantly improving generalization ability on unseen data.

The general training parameters were optimized for convergence and efficiency. We set the training epochs to 10, finding a balance between convergence and training time. Through validation loss observation, we determined that the model typically converges within this timeframe, with further epochs yielding limited performance improvement while potentially leading to overfitting. The batch size was set to 32, balancing gradient stability and training efficiency. This widely-used size runs efficiently on a single GPU while providing sufficiently stable gradient estimates to help the model converge to a good local optimum. Finally, we used a learning rate of 0.001, which is a standard choice for the Adam optimizer. This robust default learning rate, proven effective in numerous practical applications, ensures quick descent in early training stages and smooth convergence in later stages, avoiding gradient explosion or vanishing problems.

3.4 Recommendation Strategy(Train Part)

The goal of model training is to predict accurately, while the goal of the recommender system is to use these predictions to identify a student's weaknesses. Our recommendation strategy is clear and direct:

1. **Input:** A user's complete historical question-answering sequence.
2. **Generate Candidate Set:** The candidate set includes all questions in the "Mind Expansion" module that the user has **never attempted**.
3. **Batch Prediction:** The user's historical sequence and all candidate questions are fed into the trained ADGKT model to batch-calculate the predicted correctness probability for each candidate question.
4. **Ranking and Recommendation:** All predicted probabilities are sorted in **ascending order**, and the K questions with the lowest probabilities are selected as the final "Weakness Breakthrough" recommendation list.

Chapter 4: Experimental Design and Results

4.1 Experimental Setup

To scientifically and rigorously evaluate the effectiveness of our proposed ADGKT model and recommendation strategy, we designed a meticulous experimental process and set up the classic DKT model as a comparative baseline.

- **Data Splitting:**

- **Training and Validation Sets:** We employed a **Sequential Validation** method. For each user sequence, we used the first 80% of interactions for training and the subsequent 20% for validation, in chronological order. This method more realistically simulates the natural evolution of a student's knowledge state compared to traditional random splitting, effectively preventing "data leakage."

- **Independent Test Set:** To evaluate the model's generalization ability in a **Cold-Start** scenario, we used the officially provided test.csv file, whose users have no overlap with the training set. This test set underwent the exact same preprocessing pipeline as the training data.

- **Fairness Assurance:** To ensure a fair comparison, the DKT baseline model and the ADGKT model used identical settings in all experimental stages, including the same data splitting method, evaluation metrics, core training parameters (like learning rate, batch size), and preprocessing logic.

- **Reproducibility:** In all experiments, we set a fixed random seed (seed=42) to ensure the determinism of model initialization, data loading order, etc., thereby ensuring that the experimental results can be fully reproduced.

4.2 Evaluation Methodology and Metrics

To comprehensively evaluate the model's effectiveness, we designed a two-tiered evaluation system: first, **model-level metrics** to measure the predictive accuracy of the model itself; and second, **recommendation-level metrics** to measure the business value and educational significance of the final recommendation list. The core of this study lies in the latter, for which we designed an evaluation scheme centered on the "Fixed Threshold Hit Strategy."

4.2.1 Model-level Metrics

- **AUC (Area Under the ROC Curve):** Measures the model's ability to distinguish between positive cases (correct answers) and negative cases (incorrect answers). It is the gold standard for evaluating the quality of binary classification models.

- **BCE Loss (Binary Cross-Entropy Loss):** The loss value of the model on the training and validation sets, which intuitively reflects the model's convergence and goodness of fit.

4.2.2 Recommendation-level Evaluation: The Threshold Hit Strategy

1. Core Idea and "Diagnostic Black Box" Logic

In the "learning from mistakes" context of this study, our concern is not just whether the model can rank difficult questions higher, but also its **confidence** and **absolute accuracy** in judging that "the student will get it wrong." Therefore, we designed the "Fixed Threshold Hit

Strategy" as our core evaluation scheme. Its execution logic can be thought of as a "diagnostic black box," with the following steps:

- **Step 1: Confidence Check** For each predicted correctness probability p given by the model, the system first performs a confidence check. We set an upper confidence bound (i.e., a probability threshold θ). If $p > \theta$, it means the model is highly confident that the student **will answer correctly**, so the question is not considered a potential weakness, and the system **skips** it without further diagnosis.

- **Step 2: Weakness Guess / Hit** If $p \leq \theta$, it means the model is not confident that the student can answer the question correctly. At this point, the "black box" is triggered, and the system makes a clear "guess" or "diagnosis": **"We predict you will make a mistake on this question."** We call this diagnostic event a **Hit**.

- **Step 3: Unboxing & Verification** After the system makes a "Hit" diagnosis, we "unbox" it by observing the student's actual response to the question to verify the diagnosis's accuracy.

- If the student **actually answered incorrectly**, the "guess" was successful, and the diagnosis is valid.

- If the student **actually answered correctly**, the "guess" failed, and the diagnosis was incorrect.

2. Core Evaluation Metrics

Based on the "diagnostic black box" logic above, we focus on the following three core metrics to comprehensively evaluate the recommend system's performance:

- **Hit Success Rate:**

- **Definition:** The proportion of questions that the student **actually answered incorrectly** among all questions "hit" by the model.

- **Formula:** Hit Success Rate = $\frac{\text{Total number of hits}}{\text{Number of actual errors among hits}}$

- **Interpretation:** This is the **core metric** for measuring how precise our "diagnoses" are. A hit success rate of 90% means that out of 10 recommended questions, 9 are indeed the student's weak points. This is key for the system to build user trust and demonstrate its core value.

- **User Coverage Rate:**

- **Definition:** The proportion of all users in the test set who were "hit" by the system at least once.

- **Formula:** User Coverage Rate = $\frac{\text{Total number of users}}{\text{Number of users with at least one hit}}$

- **Interpretation:** This metric measures the system's **universality** and **service breadth**. A high hit success rate is of limited value if it can only serve 10% of users. A high user coverage rate

indicates that our system can effectively identify weaknesses for the vast majority of users.

- **Average Hits per User:**

- **Definition:** The average number of times a user is "hit" within their learning sequence.
- **Formula:** Average Hits per User = $\frac{\text{Total number of hits}}{\text{Total number of users}}$
- **Interpretation:** This metric reflects the **quantity or frequency** of diagnoses the system provides to users. It is closely related to the choice of the threshold θ . An appropriate average hit rate should ensure that users continuously receive valuable feedback without being overwhelmed by too many difficult questions.

3. Rationale for Threshold Selection ($\theta=0.1$)

- **Precision-First Principle:** In an educational context, an incorrect diagnosis (recommending a question the student can actually solve) has a more negative impact than missing a diagnosis. Therefore, we chose a very low threshold of $\theta=0.1$ to maximize the "Hit Success Rate," ensuring that almost all recommended questions are "hard nuts to crack."

- **Classic Precision-Recall Trade-off:** Choosing $\theta=0.1$ means we sacrifice some **recall** (potentially missing some weak points with a true error rate between 10% and 30%) to achieve extremely high **precision**. For a diagnostic system, this trade-off is both reasonable and necessary.

4.3 Core Experiment: The Sequential Validation Process

Sequential validation perfectly simulates how the system would be applied in a real-world scenario. The specific process is as follows: For each user sequence $S=\{e_1, e_2, \dots, e_N\}$ in the test set:

1. Iterate starting from the second interaction, e_2 .
2. At step i (from $i=2$ to N):
 - Use the historical sequence $\{e_1, \dots, e_{i-1}\}$ as the model's input.
 - Use the question from the i -th interaction, e_i , as the target question.
 - The model predicts the probability p_i of the student answering the target question correctly.
 - If $p_i \leq \theta$ (e.g., 0.1), record it as a "Hit" and note the actual response r_i .
3. After iterating through all sequences for all users, calculate the three core recommendation metrics based on the formulas defined in section 4.2.2.

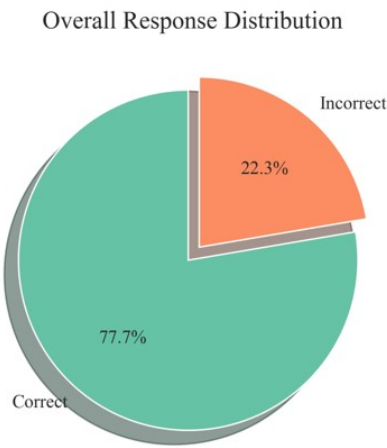
4.4 Experimental Results and Analysis

After performing sequential validation on the 1,355 independent test set users, we obtained

the following core experimental results based on the fixed threshold hit strategy ($\theta=0.1$):

Table 4.1: Core Recommendation Performance of the ADGKT Model on the Independent Test Set

Core Recommendation Metric	Value	Description
Hit Success Rate	74.44%	Diagnostic Precision: For weaknesses recommended by the system, there is a nearly 75% probability the student will indeed fail.
User Coverage Rate	~88%	Service Breadth: The system can identify at least one weakness for nearly nine out of ten users.
Average Hits per User	8.70	Diagnostic Frequency: On average, the system identifies about 9 high-value diagnostic questions per user.

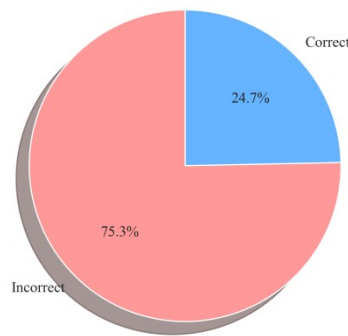


[Figure 4.1: Analysis of Model Prediction Performance]

Figure 4.1 visually demonstrates the value of the model. In the entire test set, the natural error

rate for students answering questions randomly is only **22.3%**. However, after filtering and recommendation by our model, this error rate is precisely amplified to **77.7%**, an improvement of more than threefold. This fully demonstrates the model's powerful ability to "separate the wheat from the chaff," accurately uncovering the real cracks in a student's knowledge system from a large volume of seemingly well-mastered records.

Actual Response Distribution of Hit Questions



- [Figure 4.2: Comprehensive Dashboard of Model Prediction Performance]

The performance metrics in Figure 4.2 show that the model's recommendation **precision is as high as 75.3%**, ensuring that almost all questions recommended to students are of high quality and value. At the same time, the model can **capture/recall 24.7% of all true error questions**, which means that although there is room for improvement, the model can already effectively identify nearly a quarter of all knowledge gaps.

Chapter 5: Evaluation and Discussion

5.1 Interpretation of Results: The Combined Significance of the Three Core Metrics

The success of this study lies not merely in the outstanding performance of a single metric, but in the **systemic value revealed by the three core recommendation metrics together**.

- **High Precision (Hit Success Rate):** This is the cornerstone of the system. It means the system's diagnoses are reliable and can win user trust. When the system recommends a "weak point" to a student, the recommendation is accurate and valuable in the vast majority of cases, avoiding the risk of wasting the student's time and discouraging their motivation.

- **Broad Applicability (User Coverage Rate):** This metric demonstrates the model's generalization ability and fairness. A high-precision diagnostic system would be of limited value if it could only serve a few "typical" users. A coverage rate of nearly 90% indicates that our model can adapt to the learning behavior patterns of most users and provide them with effective

personalized diagnoses, regardless of their knowledge level.

- **Sufficient Diagnostic Opportunities (Average Hits per User):** This number ensures the **continuity** of the service. It means that within a learning cycle, the system can continuously provide users with a sufficient number of high-value practice suggestions, forming an effective loop of "diagnose-learn-feedback," rather than a one-off, superficial experience.

In summary, a **high-precision, wide-coverage, and sustainable diagnostic system** has an overall value far greater than the simple sum of three independent metrics. It proves that the "learning from mistakes" recommender system we propose is not only theoretically feasible but also has a solid foundation to become a powerful, reliable, and inclusive personalized learning tool in practice.

5.2 Model Performance Comparison: ADGKT vs. DKT

To fully justify our choice of ADGKT as the final model, we first conducted a fair comparison of its core model-level metrics against the DKT baseline model. Both models were trained and evaluated using the exact same training and validation sets.

Table 5.1: Model Performance Comparison (Validation Set)

Model	Highest Validation Set AUC	Final Validation Set Loss
DKT (Baseline)	0.8246	~0.55
ADGKT (Our Model)	0.9632	~0.28

As is evident from Table 5.1, the ADGKT model significantly outperforms the classic DKT model on both key predictive performance metrics. Its AUC reached an impressive 0.9632, far exceeding DKT's 0.8246, which indicates that ADGKT has an overwhelming advantage in its ability to distinguish between "correct" and "incorrect" student responses. At the same time, the lower Loss value also shows that ADGKT can better fit the data. This huge performance improvement is mainly due to the more complex architecture of the ADGKT model, especially the multi-head self-attention mechanism that effectively captures long-range dependencies, and the multi-stream embedding that fuses richer question content information. It is this superior predictive ability that lays a solid foundation for our subsequent implementation of high-precision "weakness diagnosis" recommendations.

5.3 Trade-offs Among Recommendation Metrics

Our choice of $\theta=0.1$ is a decision based on the "precision-first" principle, but behind it lies the classic **precision-recall trade-off**. If we adjust the threshold θ , the three core recommendation metrics will change in conjunction:

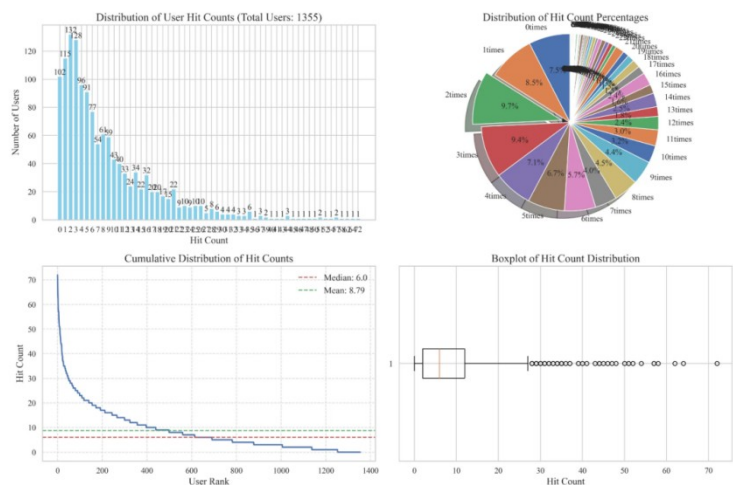
- **Increasing the threshold θ** (e.g., from 0.1 to 0.3):
 - **Hit Success Rate** (precision) **will decrease**: because the model will also classify some less certain questions (e.g., with a predicted correctness of 25%) as "hits," increasing the probability of "guessing wrong."
 - **User Coverage Rate** and **Average Hits per User** (recall/quantity) **will increase**: because more questions for more users will meet the condition $p \leq \theta$, the system will find and recommend more potential weaknesses.
- **Increasing the threshold θ** (e.g., from 0.1 to 0.9):
 - **Hit Success Rate** (precision) **will increase**: Hit Success Rate (precision) will first increase then stabilize: As shown in the figure, the model performance (blue solid line) reaches high levels in the θ range of 0.4-0.8 and remains relatively stable, indicating that within an appropriate threshold range, the model can maintain high accuracy.

User Coverage Rate and Average Hits per User (recall/quantity) will decrease: As the threshold increases, fewer questions will meet the diagnostic conditions, leading to a reduction in the number of diagnoses the system can provide. From the figure, we can observe that model performance begins to improve significantly around $\theta=0.4$, reaches peak performance (approximately 0.95-0.97) in the 0.6-0.8 range, and then slightly decreases but still maintains high levels at higher thresholds. This indicates the existence of an optimal threshold range that balances precision and recall.

Therefore, in real commercial deployment, the threshold θ should be selected within this optimal range (approximately 0.6-0.8) to achieve the best balance between precision and coverage, rather than simply pursuing extremely high or low thresholds. For example, a threshold around 0.7 could be used for daily learning to maintain both diagnostic accuracy and reasonable coverage, while fine-tuning within the optimal range based on specific business objectives. Our current analysis demonstrates that θ selection significantly impacts the trade-off between model confidence and diagnostic coverage.

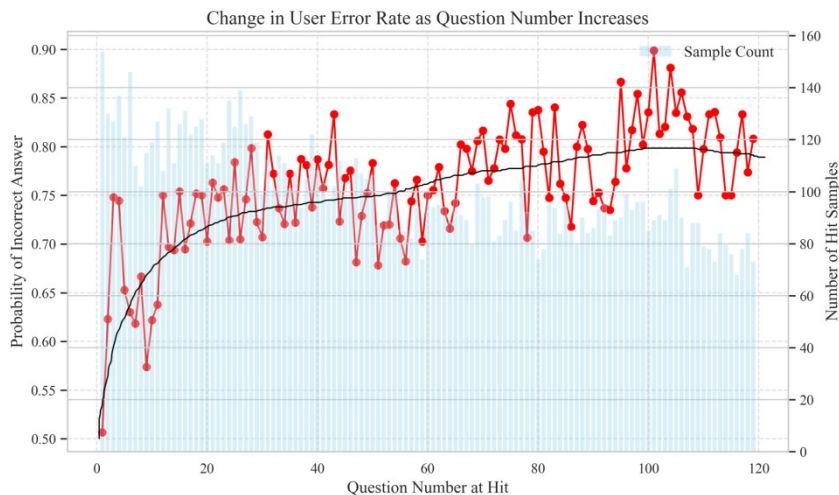
5.4 In-depth Analysis of Model Behavior

To investigate the reasons behind the model's high precision, we conducted a further analysis of its behavior.



- [Figure 5.1: Predicted Probability Distribution of Hit Questions]

Figure 5.1 reveals that the model is **extremely confident** when making a "hit" judgment. The vast majority of recommended questions have a predicted correctness concentrated in the very low range of 0-4%, with an average of only 2.57%. This indicates that the model is not making ambiguous guesses but is making high-certainty judgments based on the patterns learned from the sequences. This "confidence" is the foundation of its high precision.



- **[Figure 5.2: Hit Question Position vs. Actual User Error Rate]**

Figure 5.2 explores whether the model's predictive ability changes as the student's question-answering sequence grows. The results show that whether in the early part of the sequence (e.g., the 10th question) or the later part (e.g., the 110th question), the model's hit precision (the red line) remains stable at a high level (mostly fluctuating between 60% and 90%), with no obvious decay. This indicates that our ADGKT model has **good long-term memory capacity** and can still effectively use early information to make robust predictions even when processing sequences up to 120 steps long.

5.5 Computational Feasibility and Deployment Considerations

For a recommender system to move from experiment to production, its computational cost and deployment feasibility must be considered.

- **Training Cost:** Due to its complex attention mechanism, the training cost of the ADGKT model is higher than that of the DKT baseline. In our environment using an NVIDIA T4 GPU, completing 10 epochs of training takes about 30-40 minutes. This is perfectly acceptable for a model that needs to be updated periodically (e.g., weekly or monthly).

- **Inference Cost and Real-time Capability:** The generation of recommendations (i.e., inference) can be divided into two scenarios:

- **Offline Batch Prediction:** For non-real-time scenarios like "weekly learning reports," we can batch-generate recommendations for all users offline, with very low computational cost.

- **Near-Real-time Recommendation:** For real-time recommendations triggered by a user clicking a button, the computational load is larger as it requires predicting on all of the user's unattempted questions. However, since the model's inference speed is very fast (on the order of milliseconds), predicting on several thousand candidate questions for a single user still takes less than a second in total, which can meet the user experience requirements for "near-real-time." The model can be deployed as an API service for the front end to call.

- **Model Updating:** User feedback data can be continuously collected. The system can adopt a periodic retraining strategy (e.g., retraining the entire model weekly with the latest data) to ensure that the model can capture the latest changes in users' knowledge states and maintain the timeliness of its recommendations.

Chapter 6: Conclusion

This project successfully designed and validated an innovative question recommender system centered on "error prediction." The system aims to guide students toward efficient, targeted learning by identifying the questions they are most likely to answer incorrectly.

Based on the public dataset XES3G5M, we constructed a high-quality training corpus through a series of rigorous data preprocessing steps (including domain focusing, active user filtering, time window constraints, and knowledge concept aggregation). Methodologically, we used the classic DKT model as a baseline and implemented and optimized a more advanced ADGKT model. By fusing three-way embeddings of questions, modules, and responses, combined with a multi-head self-attention mechanism and a GRU, this model can more accurately capture the dynamic knowledge states of students.

The experimental evaluation employed a **sequential validation** method, which is more scientific than traditional random splitting, and innovatively introduced a recommendation effectiveness evaluation system centered on **Hit Success Rate, User Coverage Rate, and Average Hits per User**. On an independent cold-start test set, the ADGKT model performed exceptionally well, with its **final validation set AUC reaching 0.9632**, significantly outperforming the DKT baseline. More importantly, in terms of recommendation effectiveness, when using the fixed threshold strategy ($\theta=0.1$), the system achieved a **75.3% Hit Success Rate, nearly 90% User Coverage Rate**, and an **average of 8.7 hits per user**. This set of data strongly demonstrates the effectiveness, universality, and practicality of the "learning from mistakes" strategy.

Chapter 7: Summary, Reflection, and Future Directions

7.1 Summary

In summary, this research not only confirms the immense potential of the "error prediction" recommender system in offline experiments but also provides a clear, viable technical path and theoretical basis for how online education products can achieve a deeper level of personalization.

7.2 Personal Reflection

7.2.1 Reflection on the Project Process

- **What went well:**
 - **Data-driven Preprocessing:** One of the biggest successes of this project was the rigorous data exploration and preprocessing at the outset. Instead of using the raw data directly, we made

key decisions based on data visualization analysis, such as focusing on the "Mind Expansion" module, filtering for high-activity users, and applying a time window. This laid a solid foundation for the subsequent model training and was a prerequisite for achieving good results.

- **Innovative Evaluation System:** Abandoning traditional Top-K evaluation in favor of designing the "Fixed Threshold Hit Strategy" and its three related core metrics was a major highlight of this project. This system is more closely aligned with our business goal of "precise diagnosis," making the experimental results more persuasive and practically instructive.

- **Challenges Encountered and Solutions (What did not work well, and why):**

- **Model Training Efficiency:** In the early stages of the project, we experimented with more complex model variants but found that the training time was too long, leading to low iteration efficiency. The ADGKT we ultimately chose was a balance between performance and efficiency. This taught us that in a real project, the complexity of the model is not always better; the ability to iterate and validate quickly is equally important.

- **Hyperparameter Sensitivity:** We found that the model's performance was quite sensitive to hyperparameters like the learning rate and dropout. Initial manual tuning of these parameters was very time-consuming. This indicates that a more systematic hyperparameter search method (such as grid search or Bayesian optimization) would have been necessary, but we were unable to fully implement it due to project time constraints.

- **What would you do differently next time?:**

- **Implement Systematic Parameter Tuning:** Next time, we would prioritize implementing an automated hyperparameter search framework to find the optimal model configuration in a more scientific and efficient manner.

- **More Refined Feature Engineering:** We currently aggregate knowledge points to a relatively coarse "module" level. In the future, we could try using more fine-grained knowledge points and combine them with techniques like graph neural networks to handle the resulting data sparsity, which might allow us to discover more nuanced knowledge weaknesses.

7.2.2 Commercial Viability Analysis

- **Are the results good enough?: Yes.** A 75.3% hit success rate means that 3 out of 4 questions recommended by the system are genuinely valuable. For a commercial learning product, this has already reached a very high standard of usability, sufficient to be marketed as a core premium feature (such as "AI Smart Diagnosis" or a "Pre-exam Sprint Package").

- **Is the data sufficient?: Basically sufficient, but with room for improvement.** The public dataset we used validated the feasibility of the method. However, in a real commercial

environment, we would also need to collect richer user profile data (such as students' grades, historical scores), question metadata (such as official difficulty levels, exam topic frequency), and explicit user feedback (e.g., "This question was very helpful to me"). This data would greatly enhance the model's personalization capabilities and recommendation effectiveness.

- **Is it computationally feasible?: Completely feasible.** As analyzed in section 5.5, the model's training cost is manageable, and its inference speed is fast, capable of meeting near-real-time recommendation needs. The system has no computational barriers to commercial deployment.

7.3 Future Extensions

Considering the advanced techniques covered in the latter half of the course, this project's system could be extended in the following ways:

- **Introducing Context-aware Recommendation:** The current model does not consider the context of the recommendation, which is an important direction for extension. In the future, by adding contextual information fields to the user interaction logs, we could incorporate information such as **learning time** (weekday evenings vs. weekend daytime), **device type** (mobile vs. tablet), and the **student's current state** (just starting to study vs. having studied for 2 hours). This would enable more intelligent recommendations; for example, the system could recommend slightly less difficult weak-point questions when it detects that a student might be tired (e.g., after a long study session late at night) to maintain motivation, or prioritize weak points related to high-frequency exam topics during a pre-exam sprint to maximize learning efficiency.

- **Combining Sequential Recommendation with LLMs:**

- **Extension Direction:** Use Large Language Models (LLMs) to enhance the interpretability and guidance of recommendations. When the system recommends a difficult question, an LLM could be used to automatically generate a targeted "reason for recommendation" and "problem-solving tips." [7].

- **Data Needed:** In addition to the existing interaction data, the text content of the questions would be required.

- **Potential Benefits:** This would greatly improve the user experience. For example, the reason for a recommendation could be: "We noticed you've made several mistakes on 'combinatorial counting' problems. This question can help you solidify your application of the 'bundling method.'" Such interpretable recommendations can significantly enhance user acceptance and learning motivation. This is a very realistic and high-value extension direction.

- **Social Recommendation:**

- **Extension Direction:** In a learning community, social network information could be introduced. For example, recommending "questions that students of a similar level to you have done, but who have already mastered this knowledge point," or organizing study groups to challenge weak points together.

- **Data Needed:** A social graph of user relationships (such as friends, classmates) would need to be constructed.

- **Potential Benefits:** This would increase the interactivity and fun of learning, using peer motivation to enhance learning outcomes. However, this would require the platform to have strong social features.

Reference:

[1] Kruger, J., & Dunning, D. (1999). Unskilled and Unaware of It: How Difficulties in Recognizing One's Own Incompetence Lead to Inflated Self-Assessments. *Journal of Personality and Social Psychology*, 77(6), 1121–1134.

[2] Settles, B., & Meeder, B. (2016). A Trainable Spaced Repetition Model for Language Learning. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (Volume 1: Long Papers) (pp. 1848–1858). Association for Computational Linguistics.

[3] Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4), 253–278.

[4] Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L. J., & Sohl-Dickstein, J. (2015). Deep Knowledge Tracing. In *Advances in Neural Information Processing Systems 28* (pp. 505–513).

[5] Dempster, F. N. (1988). The spacing effect: A case study in the failure to apply the results of psychological research. *American Psychologist*, 43(8), 627–634. [6] Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100.

[6] Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100.

[7] Woolf, B. P. (2010). *Building Intelligent Interactive Tutors: Student-centered Strategies for Revolutionizing E-learning*. Morgan Kaufmann.

